# UNIT TEST REPORT

## Railway Reservation System

## 1. Introduction

This Unit Test Report documents the automated backend tests executed for the Railway Reservation System. Unit testing verifies that individual functions, modules, routes and service components behave as expected.

The test suite covers booking lifecycle and seat holds, UPI payment handling, PNR generation, refund behavior, train search API, waitlist operations, user-related functions and saved language preferences. The tests were run with the Node.js test runner.

## 2. Objectives of Unit Testing

- Verify the correctness of individual backend modules and functions.
- Detect errors and unexpected behavior early.
- Validate booking, payment, PNR and refund processing logic.
- Check waitlist state transitions and cancellation rules.
- Verify API behavior, input validation and relevant user preferences.
- Exercise concurrency and duplicate-payment protection where covered by tests.

## 3. Testing Environment

| Component | Description |
|---|---|
| Programming Language | JavaScript |
| Runtime Environment | Node.js |
| Backend Framework | Express.js |
| Testing Framework | Node.js built-in test runner (node:test) |
| Database | MongoDB |
| ODM | Mongoose |
| Test Database | railway_test (mongodb://127.0.0.1:27017/railway_test) |
| Test Command | npm test |
| Coverage Tool | c8 |
| Execution Environment | Windows CMD / Terminal |

The test run used a separate MongoDB test database configured through the MONGODB_TEST_URI environment variable.

## 4. Unit Test Cases

The following are representative test scenarios visible in the test execution evidence. The automated suite contains 81 tests across 13 suites; this table is not an exhaustive listing.

### 4.1 Booking Lifecycle and Payment

| Test ID | Test Scenario | Expected Result | Actual Result | Status |
|---|---|---|---|---|
| UT-001 | Booking hold seats while status is PENDING_PAYMENT and has no PNR/server-side fare | Seats are held; booking remains pending and PNR/fare are not finalized | Test passed (shown in execution log) | PASS |
| UT-002 | Successful UPI payment confirms booking and generates a PNR | Booking is confirmed and a PNR is generated | Test passed (shown in execution log) | PASS |
| UT-003 | Invalid UPI ID is rejected and booking is not confirmed | Payment attempt is rejected; booking remains unconfirmed | Test passed (shown in execution log) | PASS |
| UT-004 | Declined payment keeps booking unconfirmed and allows retry | Booking remains unconfirmed and retry is permitted | Test passed (shown in execution log) | PASS |
| UT-005 | Paying twice does not create a second payment or PNR | Only one payment/PNR is created for the booking | Test passed (shown in execution log) | PASS |
| UT-006 | Concurrent payments confirm | Concurrent payment requests | Test passed (shown in | PASS |

| | | | | |
|---|---|---|---|---|
| | exactly once | result in exactly one confirmation | execution log) | |
| UT-007 | Canceling a confirmed ticket releases the seat | Cancellation succeeds and the seat is released | Test passed (shown in execution log) | PASS |

### 4.2 Waitlist Monitoring

| Test ID | Test Scenario | Expected Result | Actual Result | Status |
|---|---|---|---|---|
| UT-008 | A waitlisted ticket cannot be cancelled twice | Repeated cancellation is prevented | Test passed (shown in execution log) | PASS |
| UT-009 | Ticket remains waitlisted when train departure is cancelled with a full refund | Waitlist status and refund behavior follow the tested rule | Test passed (shown in execution log) | PASS |

Other test suite areas reported in the project include train search API, PNR, refund, saved language preferences, booking routes and user-related functions. Their individual case details are not reproduced here because the supplied screenshots do not show the full test-case listing.

## 5. Test Execution Summary

| Parameter | Result |
|---|---|
| Total Tests Executed | 81 |
| Test Suites | 13 |
| Tests Passed | 81 |
| Tests Failed | 0 |
| Tests Cancelled | 0 |
| Tests Skipped | 0 |
| Duration | Approximately 6.89 seconds |
| Overall Result | PASS |

All 81 automated tests completed successfully. The terminal output also displays Mongoose deprecation warnings regarding the 'new' option in findOneAndUpdate() and findOneAndReplace(); these warnings did not cause test failures.

## 6. Testing Evidence

The following screenshots supplied for this report show the test execution, final summary and c8 coverage results.

Evidence 1: npm test output showing booking lifecycle and payment tests.

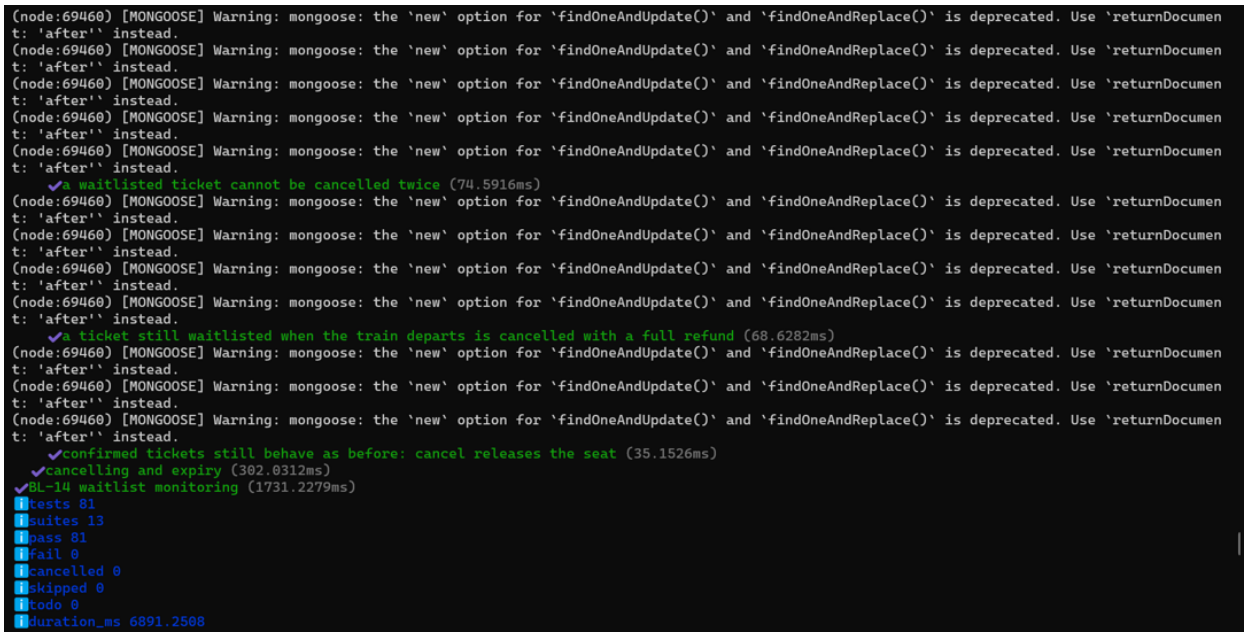
```
C:\Users\nagav\Desktop\RAILWAY-RESERVATION-SYSTEM-\backend>set MONGODB_TEST_URI=mongodb://127.0.0.1:27017/railway_test

C:\Users\nagav\Desktop\RAILWAY-RESERVATION-SYSTEM-\backend>npm test

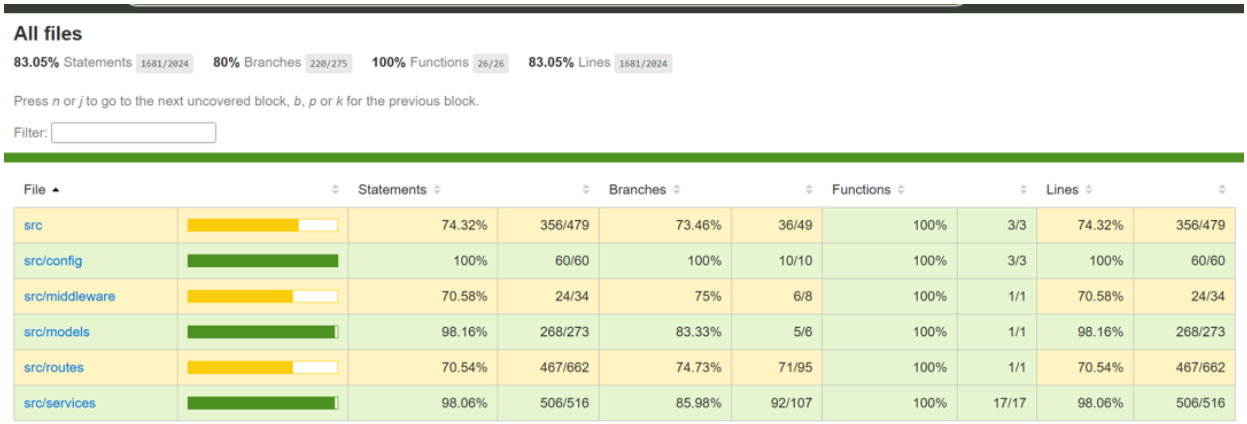
> railway-reservation-backend@1.0.0 test
> node --env-file=.env --test --test-concurrency=1

◇ injected env (0) from .env
(node:68444) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
(Use `node --trace-warnings ...` to show where the warning was created)
▶ BL-11/12/13 booking lifecycle
  ✔booking holds seats, is PENDING_PAYMENT, has no PNR and a server-side fare (148.4769ms)
(node:68444) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
(node:68444) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
  ✔successful UPI payment confirms booking and generates a PNR (TC_05, TC_06) (35.4188ms)
(node:68444) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
  ✔invalid UPI ID is rejected and booking is not confirmed (23.6386ms)
(node:68444) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
(node:68444) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
  ✔declined payment keeps booking unconfirmed and allows retry (UC-04 4a-4c) (41.5949ms)
(node:68444) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
(node:68444) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
  ✔paying twice does not create a second payment or new PNR (36.4299ms)
(node:68444) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
(node:68444) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
(node:68444) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
(node:68444) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
  ✔concurrent payments confirm exactly once (45.837ms)
(node:68444) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
(node:68444) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
```

Evidence 3: Final test summary: 81 tests, 13 suites, 81 passed, 0 failed, 0 skipped.



Evidence 4: c8 HTML coverage overview for all files.



All files

83.05% Statements 1681/2024 80% Branches 220/275 100% Functions 26/26 83.05% Lines 1681/2024

Press n or j to go to the next uncovered block, b, p or k for the previous block.

Filter:

| File | Statements | | Branches | | Functions | | Lines | |
|---|---|---|---|---|---|---|---|---|
| src | 74.32% | 356/479 | 73.46% | 36/49 | 100% | 3/3 | 74.32% | 356/479 |
| src/config | 100% | 60/60 | 100% | 10/10 | 100% | 3/3 | 100% | 60/60 |
| src/middleware | 70.58% | 24/34 | 75% | 6/8 | 100% | 1/1 | 70.58% | 24/34 |
| src/models | 98.16% | 268/273 | 83.33% | 5/6 | 100% | 1/1 | 98.16% | 268/273 |
| src/routes | 70.54% | 467/662 | 74.73% | 71/95 | 100% | 1/1 | 70.54% | 467/662 |
| src/services | 98.06% | 506/516 | 85.98% | 92/107 | 100% | 17/17 | 98.06% | 506/516 |

## 7. Code Coverage Summary

| Coverage Metric | Result |
|---|---|
| Statements | 83.05% (1681/2024) |
| Branches | 80.00% (220/275) |
| Functions | 100.00% (26/26) |
| Lines | 83.05% (1681/2024) |

The c8 overview also shows source-folder coverage by component. Coverage reflects the code exercised by the test suite and does not imply that every possible behavior or end-to-end UI workflow has been tested.

## 8. Conclusion

Unit testing was successfully performed on the Railway Reservation System backend using Node.js node:test with MongoDB test database integration. The latest execution resulted in 81 passed tests, zero failed tests and zero skipped tests. The c8 report measured 83.05% statement/line coverage, 80.00% branch coverage and 100.00% function coverage. These results indicate that the tested backend behaviors operated as expected under the test conditions. Mongoose deprecation warnings were present but did not affect the successful test outcome.